Quest 9: The Sulphurprise

Lord of the Wings · a plant whose branches keep splitting in three

> This guide uses the official puzzle text. The story is summarised and the rules are restated in plain words. It explains what the quest asks and the logic behind my solutions, not the code itself. Every number in the examples and traces was checked by running my solutions on the sample data (plus small instrumented copies to print intermediate values).

THE QUESTION

A plant starts as one branch that grows one element per second; whenever a branch reaches the odd length *growth*, it splits into two halves around a branching point and sprouts a third, new branch. How many seconds until the plant reaches its *limit*?

INPUT

One plant per line: `id=…, growth=…, limit=…`.

OUTPUT

The sum of the stopping times of all plants.

WHAT CHANGES BETWEEN PARTS

Part I limits the total **size** (all elements). Part II limits the number of **leaves** (branch tips). Part III is Part II with huge numbers, so you can't simulate second by second any more.

The story

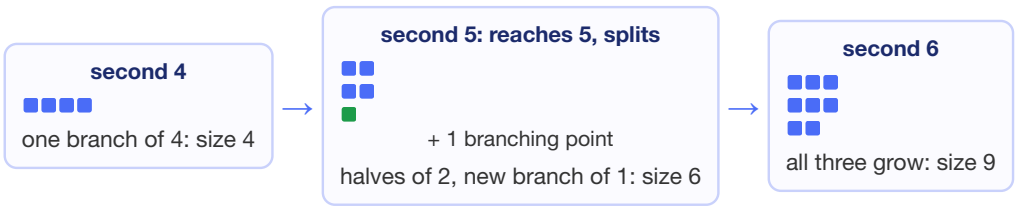
While the Ents grow their army, the team plans to sneak into Nulldor through the sewers and blow it up from inside. The weapon is a plant that grows explosively when it touches sulphur, and Nulldor is full of sulphur. Before carrying it in, the hero wants to know exactly how fast each variety grows, first by size, then by number of growing tips, and finally for much bigger varieties.

Understanding the question

The rules

- After second 1 there is one branch, 1 element long. The seed itself never counts.
- Every second, **every** branch grows by one element, all at the same time.
- When a branch reaches exactly *growth* elements (always odd), it splits at that moment: the middle element becomes a **branching point**, the two sides become two branches of (*growth* − 1) / 2 each, and a **new branch**, already 1 element long, starts from the branching point.
- The new branch doesn't grow again in the same second. From the next second, all three grow, and each can split later.
- The **size** is all branch elements plus all branching points. A **leaf** is a growing tip: there is 1 at the start and each split adds exactly 1 (the new branch's tip). So leaves = 1 + number of splits. Note this is *not* the number of branches: a split turns one branch into three, but adds only one leaf.
- The time for a plant is the first second at the end of which the measured value is ≥ *limit*.

One split, slowly



growth = 5. A split adds two elements at once: the branching point and the new 1-element branch (green).

Extra examples of my own (run through copies of my solution)

growth	limit	Part I time	Why
3	1	1	The plant already has size 1 after the first second, so it stops immediately.
3	4	3	Size goes 1, 2, 4: the split in second 3 jumps from 2 straight past 3 to 4, which reaches the limit exactly.
5	6	5	Sizes 1, 2, 3, 4, 6. The limit is reached in the split second itself.

Shared approach: reading the notes

Each line gives one plant's id, growth and limit. The solution reads the values by position (id first, then growth, then limit) and ignores the names, so it relies on that order.

PART I Limit the size

Count every element of every branch plus every branching point. Find the first second where that total is ≥ *limit*, and add up these times.

Worked examples

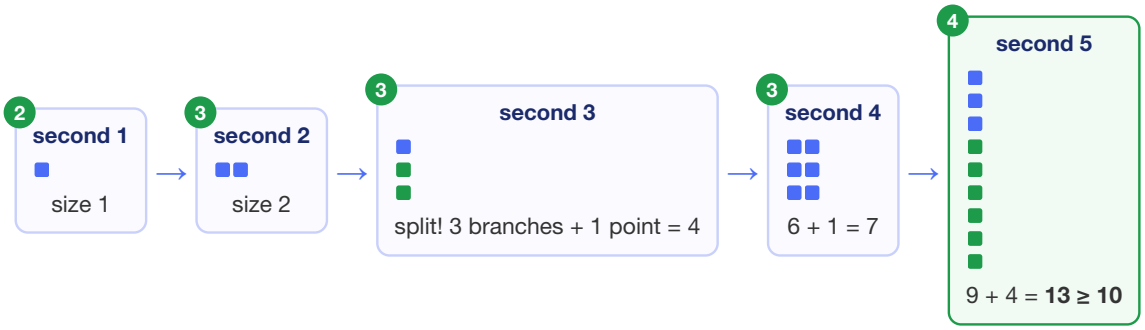
Plant	growth	limit	Sizes, second by second	Time
1	3	10	1, 2, 4, 7, **13**	5
2	5	20	1, 2, 3, 4, 6, 9, 12, 17, **25**	9
3	7	30	1, 2, 3, 4, 5, 6, 8, 11, 14, 17, 22, 29, **37**	13
			sum	**27**

Step-by-step solution

1. Read each plant's *growth* and *limit*.
2. Start with one branch of length 1 and no branching points, at second 1.
3. While (sum of branch lengths + branching points) is below the limit: grow every existing branch by 1; any branch that reaches *growth* becomes a half, and a second half plus a new 1-branch are added (not grown this second); count a branching point. Add a second.
4. The second when it stops is the plant's time; add up all plants.

How my solution works

Simulate **second by second**, keeping every branch's length and a count of branching points. Plant 1 (growth 3, limit 10):



Dots match the step numbers above. Green = branches created by a split this second (they don't grow until the next second). A split adds two elements at once: the branching point and the new 1-element branch.

Why the green branches wait (fragile): new branches are added to the list being grown, and the growing step fixes how many branches to visit *before* it starts, so they're skipped until the next second. If it instead ran to the end of the growing list, the times would become 4, 8, 11 instead of 5, 9, 13 (tested in a copy).

PART II Limit the leaves

Same growth, but now the limit is on the number of **leaves** (branch tips). Every split adds one leaf. Stop at the first second with leaves ≥ *limit*.

Worked examples

Plant	growth	limit	Leaves, second by second	Time
1	3	10	1, 1, 2, 2, 5, 5, **14**	7
2	5	20	1, 1, 1, 1, 2, 2, 2, 4, 5, 5, 9, 13, 14, **22**	14
3	7	30	1 ×6, 2 ×4, 4, 4, 5, 5, 9, 9, 13, 13, 22, 22, **34**	21
			sum	**42**

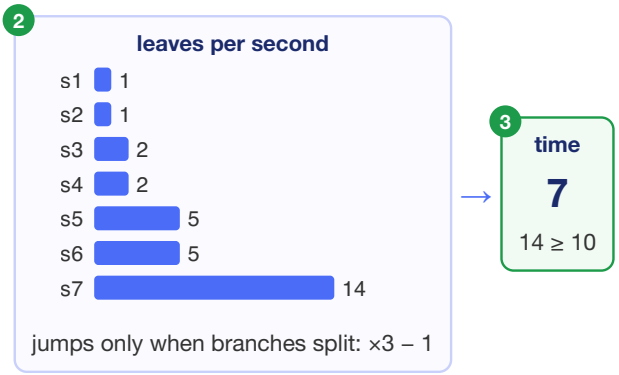
growth	limit	Part II time	Why
9	2	9	The first split happens when the first branch reaches 9, in second 9.
3	1	1	One leaf from the start.
3	5	5	Leaves 1, 1, 2, 2, 5: in second 5 three branches split at the same time, +3 at once.

Step-by-step solution

1. Do the same simulation as Part I.
2. Keep a leaf counter that starts at 1 and goes up by 1 for each split.
3. Stop when leaves ≥ limit instead of checking the size.

How my solution works

The same simulation, plus a **leaf counter** (start 1, +1 per split); stop when it reaches the limit. Plant 1:



Dots match the step numbers above. When it stops, the list holds 27 branches for just 14 leaves.

> **Cost:** one step per second, and each step touches every branch, so the work is about (seconds × branches). That's fine here (the real input runs in about 0.1 s), but it's exactly what Part III makes impossible: with a leaf limit in the billions, the list alone would need billions of entries.

PART III The same rules, at scale

Exactly the rules of Part II, with much larger *growth* and *limit* values. The puzzle gives no new example: on the sample, Part III must give the Part II answer, 42 (and my solution does).

Two key ideas: branches of the same length behave the same, so keep only a **count per length** (lengths stay below *growth*, so this table is tiny); and nothing changes between splits, so **jump** straight to the next one.

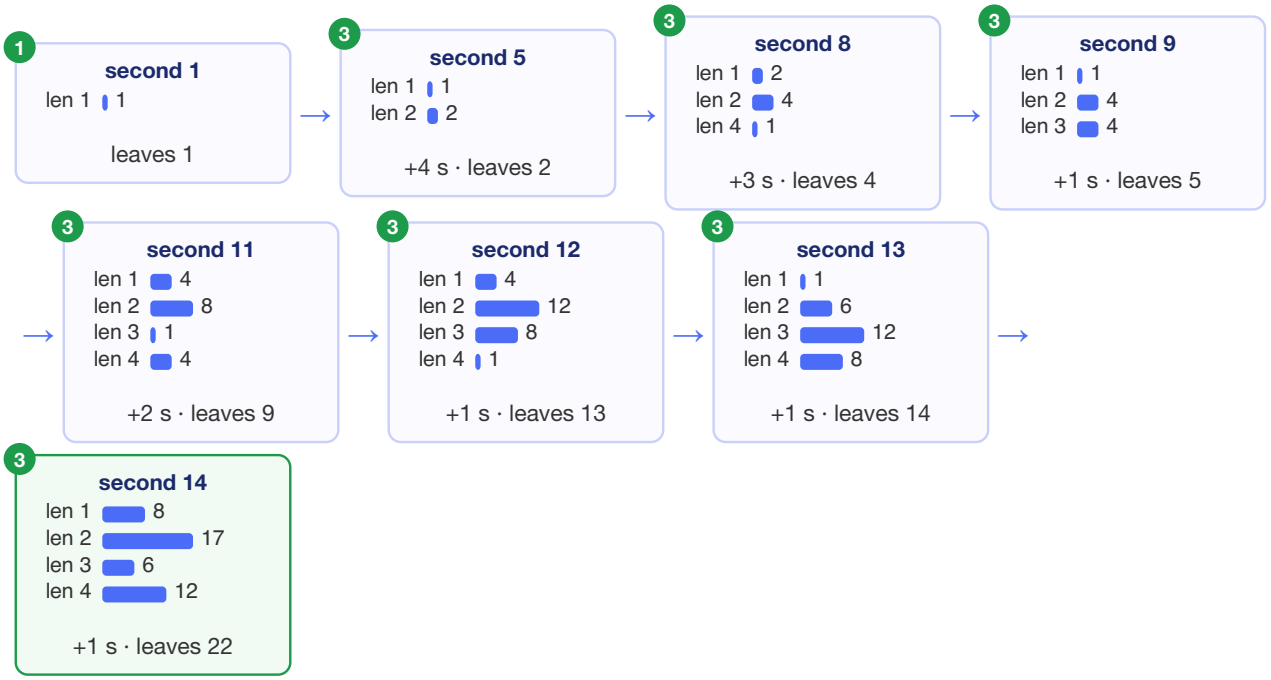
Step-by-step solution

1. Store how many branches have each length, not every branch.
2. Find the longest length; the next split is *growth* − longest seconds away. Jump the clock there.
3. Add the jump to every length. Groups that reach *growth* add their count to the leaves and turn into 2 × count halves plus count new 1-branches.

↻ *Repeat until leaves ≥ limit; keep every number exact (they get huge), and add up the times.*

How my solution works

Keep a **count per length**. Find the longest branch, **jump** the clock straight to when it reaches *growth*, add the jump to every length, split the groups that reached *growth*, and merge equal lengths. Plant 2 (growth 5, limit 20):



Dots match the step numbers above. Seven jumps instead of 13 seconds; the leaf counts are exactly the puzzle's. For growth 3 every length is always 1, so the counts are just 1×3, 1×9, 1×27.

Why it's safe to jump: all branches grow at the same speed, so none can split before the longest one, and leaves only change at a split. On the real input all plants together need about 15,000 jumps, with at most 38 different lengths, in under 0.1 s.

> **Exact numbers:** the counts get huge, so everything is kept as exact whole numbers of unlimited size (ordinary numbers are only exact up to about 9×10^{15}).

Summary

Part	Limit on	State kept	Time step	Sample
I	size (elements + branching points)	list of every branch length	1 second	27
II	leaves (1 + number of splits)	same list, plus a leaf counter	1 second	42
III	leaves, huge values	count per length (exact big numbers)	jump to the next split	42

Key ideas: things that behave the same can be **grouped and counted** instead of stored one by one (the same trick as the classic "lanternfish" style puzzles), and when nothing changes between events, **jump from event to event** instead of ticking every second.